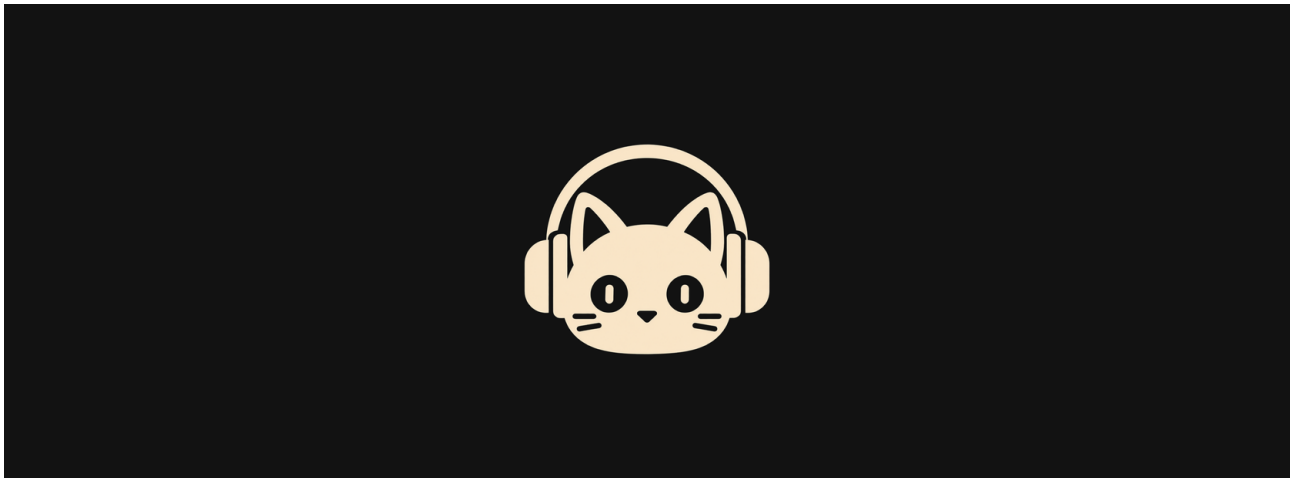


MOON PLAYER

AI Studio Development Brief & Asset Guide

An Android-first, offline MP3 player with a terminal-inspired interface, adaptive album-art accent colors, and a minimalist cat-with-headphones identity.

Purpose — Give an AI coding assistant a clear product specification, implementation plan, visual direction, and acceptance criteria for building Moon Player.

Design reference — The user-provided music-player screenshot guides the overall screen composition. Moon Player must use original branding and its own implementation rather than copying AIMP assets or identity.

Version 1.0 • 22 October 2026

1. Product summary

Moon Player is a local-first Android music player that plays MP3 files stored on the device. It combines familiar full-featured player controls with a terminal-inspired visual style. No account, streaming service, backend, ads, or telemetry is required for the first release.

The primary design goal is to preserve the reference screen's hierarchy: top app bar, large square cover, centered metadata, seek bar, prominent play/pause button, and a compact secondary-control row.

2. Main player screen

- Top bar: hamburger on the left, centered playlist/context title and track index below, overflow menu on the right.
- Large square cover art with rounded corners, thin border, and a default mascot cover when artwork is missing.
- Track title, artist, and album/collection name centered beneath artwork.
- Seek bar with elapsed and total time; playback speed label nearby.
- Primary row: bookmark/queue marker, previous, large play/pause, next, favorite.
- Secondary row: equalizer, shuffle, sleep timer, repeat, and optional A–B loop.

Use responsive constraints and safe-area insets rather than hard-coded screen coordinates. Keep the player usable on small phones, large font settings, and landscape.

3. Navigation and queue

Hamburger menu: Library, Folders, Playlists, Albums, Artists, Songs, Favorites, Recently played, Equalizer, Sleep timer, Scan library, Settings, About.

A right-edge inward swipe opens the current queue/playlist panel. Include an alternate visible button/menu action for accessibility and users who disable gestures. Avoid conflicts with Android's system Back gesture.

- Queue features: current-track highlight, drag reorder, remove item, play next, add to queue, clear queue with confirmation, track duration.
- Playlist features: create, rename, reorder, add/remove songs, and delete a playlist without deleting audio files.

4. Playback and AIMP-inspired feature scope

Aim for a robust feature set common to advanced local music players, while retaining original Moon Player branding and UI. Prioritize reliable core playback before advanced audio effects.

- Play/pause, previous/next, seek, queue, shuffle, repeat off/all/one, resume state, playback speed.
- Background playback, Android media notification, lock-screen controls, Bluetooth/headset media buttons, audio focus, and pause on headphone disconnect.
- Sleep timer, stop at end of current track, favorites, recently played, play counts, bookmarks, search and sorting.
- Equalizer integration where supported by Android/device audio effects; disclose device limitations.
- Optional later features: crossfade, gapless tuning, A–B loop, tag editing, multi-select, and playlist backup/export.

5. MP3 library and metadata

Use MediaStore to discover local audio and filter strictly to MP3 files for version 1.0. Apply Android-version-appropriate media permissions and explain permission use before scanning.

- Read title, artist, album, duration, track number, year, and embedded cover art when available.
- Fallback title from filename; show sensible “Unknown artist/album” labels and a default cover.
- Support rescan; handle moved/deleted files, corrupt tags, missing artwork, empty files, and permission denial without crashing.
- Never upload audio. Do not edit tags or delete source files unless the user explicitly confirms a clearly labelled action.

6. Visual design system

Mood: minimal, calm, slightly technical, warm rather than neon. Use terminal cues in small labels, separators, and typography; do not make the whole UI resemble an actual shell or sacrifice readability.

- Terminal background: #121212
- Raised panel: #1B1918
- Default warm ivory accent: #E8D0AE
- Muted text: #8A8177
- Borders/dividers: #34302D

Extract an accent from the current track’s cover art, cache it, and adjust it for contrast. Provide a fixed accent override and warm-ivory fallback. Use readable sans-serif for main metadata and optional monospace for small technical labels.

Touch targets should be at least 48 dp. Provide content descriptions, scalable text, TalkBack support, strong contrast, and reduced-motion-aware transitions.

7. Recommended Android architecture

- Native Android, Kotlin, Jetpack Compose, and Material 3 customized with Moon Player theme tokens.
- AndroidX Media3 ExoPlayer for audio playback with MediaSessionService for background playback.
- MediaStore for local audio discovery; Room for playlists, favorites, play history, and queue persistence; DataStore for settings.
- Coil or equivalent for artwork loading/caching; cache extracted accent colors.
- Architecture: Compose UI → ViewModels/state holders → domain use cases → repositories → MediaStore/Room/Media3.

Recommended starting minimum: Android 8.0 / API 26+, unless broader device support is a firm requirement. Target the current Play Store API level at release. No backend is required; avoid internet permission unless a documented future feature needs it.

8. Suggested data model

Track: MediaStore ID, content URI, title, artist, album, duration, track number, year, MIME type, filename, cover URI, date added.

Playlist: ID, name, created/updated timestamps, sort order. PlaylistTrack: playlist ID, track reference, position.

PlaybackState: queue IDs, current index, position, repeat mode, shuffle state, playback speed.

TrackStats: track reference, play count, last played time, favorite flag. Settings: theme/accent mode, fixed accent, default speed, resume behavior, headphone behavior, gesture state, time display mode.

9. Settings

- Theme: terminal dark by default; optional light theme later.

- Accent: dynamic from album art, warm ivory, or user-selected color.
- Library scan/rescan, folder exclusions if supported, sorting and default library view.
- Playback speed, resume-on-launch, pause-on-unplug, elapsed/remaining time display.
- Right-edge gesture toggle and sensitivity; playlist backup/export and restore.
- About, privacy statement, license notices, app version and build info.

10. Acceptance checklist

- Fresh install explains permissions and scans local MP3s; unsupported formats are not shown.
- Core controls and queue actions work; background playback continues with screen off.
- Notification, lock screen, headset, and Bluetooth controls work.
- Right-edge swipe opens queue and does not make system navigation unusable.
- Playlists, favorites, history, queue, and preferences persist after restart.
- Dynamic accent changes per track, remains readable, and falls back when artwork is absent.
- Sleep timer, repeat, shuffle, speed, and equalizer handle unsupported states gracefully.
- No crash on denied permissions, corrupt metadata, missing files, or audio focus changes.
- App icon and in-app palette match the supplied asset pack; accessibility and responsive layouts are checked.

11. Build roadmap for AI Studio

- Milestone 1 — Project scaffold, dependency plan, design tokens, navigation, permissions, MediaStore scan.
- Milestone 2 — Media3 service, player screen, notification, audio focus, persistent queue.
- Milestone 3 — Songs/albums/artists/folders, search, sorting, favorites, history.
- Milestone 4 — Playlist management, right-edge queue panel, queue reordering.
- Milestone 5 — Dynamic accent, sleep timer, speed, repeat/shuffle, equalizer integration.
- Milestone 6 — Accessibility, lifecycle tests, small-screen QA, error states, release build and README.

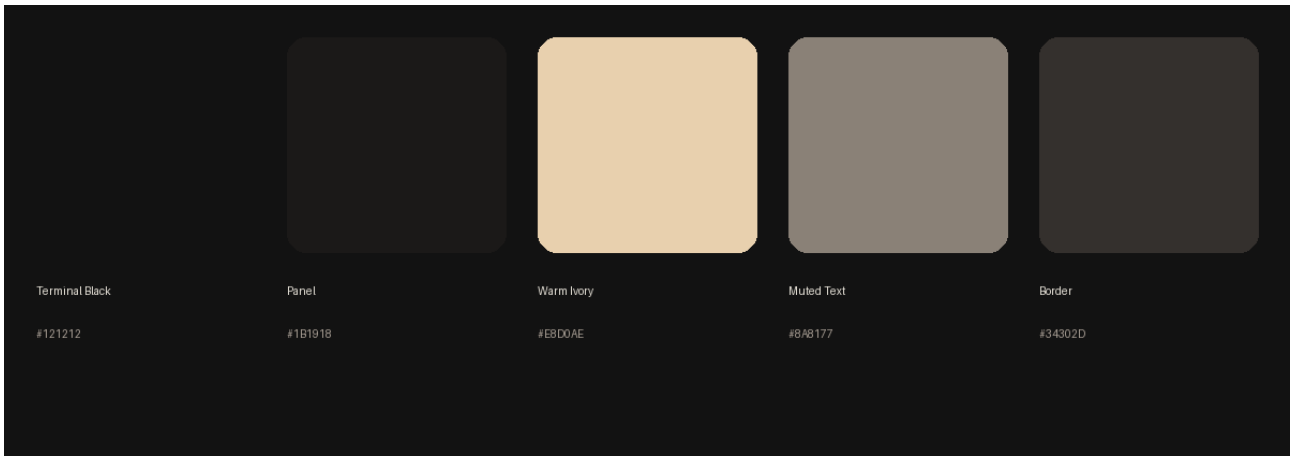
12. Instructions to the coding assistant

Start by proposing the file tree, dependencies, and architecture. Then implement one milestone at a time in compilable increments. For each change, provide full file paths and complete contents, not snippets with omitted sections. Build and test after each milestone. Use current stable Android/Media3 APIs and verify uncertain API details against current documentation. Do not leave TODO placeholders for core features. If an Android platform restriction prevents a feature, explain it and implement the least surprising fallback.

Source of truth: this PDF, the accompanying Moon_Player_Development_Spec.md, the included brand assets, and the supplied reference screenshot. Keep Moon Player MP3-only, local-first, and Android-native for the initial release.

13. Included brand assets

The companion ZIP contains the following ready-to-use assets. PNGs are raster exports; retain the transparent master where possible and use adaptive foreground/background assets for Android adaptive icons.



- moon_player_logo_transparent_1024.png — transparent mascot/logo master.
- moon_player_app_icon_1024.png / 512.png / 192.png — app icon exports on terminal-black background.
- moon_player_adaptive_foreground.png — transparent foreground layer for Android adaptive icons.
- moon_player_adaptive_background.png — solid adaptive icon background.
- moon_player_palette.png — palette reference.
- Moon_Player_Development_Spec.md — complete handoff specification in editable Markdown.

Note: generated logo is a starter brand asset. Before a public release, review it at small sizes and refine vector paths if a perfectly crisp SVG is required.